

GitHub Actions – Advanced CI-CD Concepts

Matrix Builds & Parallel Jobs

Consider a Spring Boot application with a normal CI pipeline:

```
Code
↓
Build
↓
Test
↓
Package
```

Suppose the application is currently tested using:

```
Ubuntu + Java 21
```

If the pipeline passes, it proves that the application works in **that particular environment**.

```
Application
|
↓
Ubuntu + Java 21
|
PASS
```

It does not automatically prove that the application works with:

- Java 17
- Windows
- macOS
- Other supported runtime versions

If an application or library officially supports multiple environments, ideally the CI pipeline should verify all of them.

What is a Matrix Build?

Suppose a Java library supports:

```
Java 17  
Java 21
```

One approach would be to manually create two almost identical jobs:

```
jobs:  
  
test-java-17:  
  runs-on: ubuntu-latest  
  steps:  
    ...  
    java-version: 17  
    ...  
    ./mvnw test  
  
test-java-21:  
  runs-on: ubuntu-latest  
  steps:  
    ...  
    java-version: 21  
    ...  
    ./mvnw test
```

This works, but almost the entire job has been duplicated.

Only one value changed:

```
17 → 21
```

This is exactly the problem that **Matrix Builds** solve.

A matrix build means:

Run the same job multiple times using different values.

Instead of manually defining multiple jobs, we define the possible values:

```
java: [17, 21]
```

GitHub Actions generates the corresponding jobs automatically.

Example

```
jobs:

test:

strategy:
matrix:
java: [17, 21]

runs-on: ubuntu-latest

steps:

-uses: actions/checkout@v4

-uses: actions/setup-java@v4
with:
distribution: temurin
java-version: ${ matrix.java }

-run: ./mvnw test
```

Conceptually:

```
One Job Definition
  ↓
  Matrix
 /    \
Java 17 Java 21
```

GitHub creates a separate job for every matrix value.

Inside the workflow, the current value can be accessed using:

```
${{ matrix.java }}
```

Why Is It Called a Matrix?

The idea becomes clearer when there is more than one variable.

Suppose the application supports:

```
strategy:
matrix:
os:[ubuntu-latest, windows-latest]
java:[17,21]
```

There are two dimensions:

Operating System

```
Ubuntu
Windows
```

Java Version

```
17
21
```

GitHub generates every possible combination:

Operating System	Java
Ubuntu	17
Ubuntu	21
Windows	17
Windows	21

Therefore:

```
2 Operating Systems × 2 Java Versions
=
4 Jobs
```

Visually:

	Java 17	Java 21
Ubuntu	✓	✓
Windows	✓	✓

We did not manually write four jobs.

We described the **dimensions of the testing environment**, and GitHub generated the combinations.

Parallel Jobs

Matrix jobs usually do not depend on each other.

For example:

```
Ubuntu + Java 17
Ubuntu + Java 21
Windows + Java 17
Windows + Java 21
```

Testing Java 21 does not require Java 17 to finish first.

Similarly, Windows testing does not depend on Ubuntu testing.

Running them sequentially would look like:

```
Job 1
↓
Job 2
↓
Job 3
↓
Job 4
```

That wastes time.

Instead, independent jobs can run in parallel:

```
GitHub Actions —┬─ Ubuntu + Java 17
                  ├── Ubuntu + Java 21
                  ├── Windows + Java 17
                  └─ Windows + Java 21
```

GitHub Actions normally attempts to run matrix jobs concurrently, depending on runner availability.

Matrix Build vs Parallelism

These concepts are related but different:

- **Matrix Build** → generates multiple variations of the same job.
- **Parallelism** → allows independent jobs to execute at the same time.

A matrix often naturally creates jobs that can run in parallel.

Sequential Jobs

Not every job should run independently.

Suppose deployment should happen only after testing succeeds.

```
jobs:  
  
  test:  
    ...  
  
  deploy:  
    needs: test
```

Now the dependency becomes:

```
TEST  
↓  
DEPLOY
```

The `deploy` job cannot begin until the `test` job finishes successfully.

So:

```
Independent work → Parallel  
  
Dependent work → Sequential
```

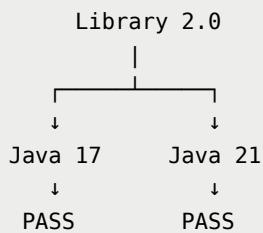
Why Companies Use Matrix Builds

Matrix builds are especially useful when building software that promises compatibility across multiple environments.

For example, imagine a Java SDK supports:

```
Java 17  
Java 21
```

Before releasing version `2.0`, CI can automatically verify both:



The compatibility claim is now backed by automated testing.

Matrix builds are useful for testing across combinations such as:

- Multiple Java versions
- Multiple Node.js versions
- Multiple operating systems
- Different database versions
- Different dependency versions

Excluding Matrix Combinations

Sometimes a particular combination is not supported.

Example:

Java 17 + Ubuntu	✓
Java 21 + Ubuntu	✓
Java 17 + Windows	✓
Java 21 + Windows	✗

We can keep the matrix and simply exclude the unsupported combination.

```
strategy:
matrix:
os: [ubuntu-latest, windows-latest]
java: [17, 21]

exclude:
```



```
-os: windows-latest  
java:21
```

GitHub now generates:

```
Ubuntu + Java 17  
Ubuntu + Java 21  
Windows + Java 17
```

but skips:

```
Windows + Java 21
```

Secrets Management

CI/CD pipelines often need sensitive information.

Examples include:

```
Database password  
AWS credentials  
Docker registry password  
API key  
SSH private key
```

These values should never be written directly inside the workflow.

Bad example:

```
env:  
DB_PASSWORD: mySuperSecret123
```

Another bad example:

```
-run: aws configure set aws_secret_access_key abcdef123456
```

The workflow file is stored in Git, and Git is specifically designed to preserve history.

Even if the secret is later removed from the YAML file, it may still exist in:

```
Git commit history
Old branches
Forks
Clones
Pull requests
```

The fundamental principle is:

Source code and credentials have completely different lifecycles.

Code is supposed to be:

```
Stored
Shared
Reviewed
Versioned
```

Credentials should be:

```
Hidden
Restricted
Rotatable
Replaceable
```

Therefore:

Code	Secret
↓	↓
Git Repository	Secret Store

GitHub Secrets

GitHub provides a separate mechanism for storing sensitive values used by GitHub Actions.

Examples:

```
AWS_ACCESS_KEY_ID
AWS_SECRET_ACCESS_KEY
DB_PASSWORD
DOCKER_PASSWORD
```

Instead of writing:

```
password: superSecret123
```

we reference the secret by name:

```
password: ${ secrets.DB_PASSWORD }
```

The workflow contains the **name of the secret**, not its actual value.

Conceptually:

```
GitHub Repository
  |
  | workflow requests
  ↓
DB_PASSWORD
  |
  ↓
GitHub Secret Store
  |
  ↓
Actual Password
  |
  ↓
Runner
```

GitHub supports secrets at different levels, including:

- Repository secrets
- Organization secrets

- Environment secrets

Creating and Using a Repository Secret

Suppose a deployment requires a database password.

In the GitHub repository:

```
Settings
  ↓
Secrets and variables
  ↓
Actions
  ↓
New repository secret
```

Create:

```
Name:
DB_PASSWORD

Value:
some-password
```

The workflow can then use it:

```
jobs:

  deploy:
    runs-on: ubuntu-latest

    steps:

      -name: Deploy
      env:
        DB_PASSWORD: ${ secrets.DB_PASSWORD }
```

```
run: |  
    ./deploy.sh
```

During the job:

```
DB_PASSWORD
```

is available to the runner as an environment variable.

The actual password does not have to be stored in the YAML file.

Secrets vs Variables

Not every configuration value needs to be secret.

Examples:

```
JAVA_VERSION=21  
ENVIRONMENT=production  
SERVER_PORT=8080
```

Someone knowing that the application uses Java 21 or port 8080 usually does not create a security problem.

GitHub therefore provides both **variables** and **secrets**.

```
Normal Configuration  
  ↓  
  Variables  
  
Sensitive Configuration  
  ↓  
  Secrets
```

Variable

```
${{ vars.JAVA_VERSION }}
```

Secret

```
${{ secrets.DB_PASSWORD }}
```

A useful rule is:

If exposing the value publicly would create a security problem, treat it as a secret.

Environment Secrets

Most real applications have multiple deployment environments.

For example:

```
Development
Staging
Production
```

Each environment may require a different database password, API key, server configuration, or cloud credential.

Production credentials should also not necessarily be available to every workflow.

GitHub **Environments** allow secrets and protection rules to be associated with a particular deployment target.

Conceptually:

```
Deploy Job
  |
  ↓
Production Environment
  |
  ├── Secrets
  |
  └── Protection / Approval Rules
```

Creating Multiple Environments

Inside the repository:

```
Settings
  ↓
Environments
  ↓
New environment
```

Create:

```
development
staging
production
```

Then open each environment and add an environment secret:

```
Environment secrets
  ↓
Add secret
```

We can use the **same secret name** in every environment:

```
DB_PASSWORD
```

while providing different values.

Example:

```
development
└─ DB_PASSWORD = dev-password-123

staging
└─ DB_PASSWORD = staging-password-456

production
└─ DB_PASSWORD = prod-password-789
```

The advantage is that the workflow does not need different variable names such as:

```
DEV_DB_PASSWORD  
STAGING_DB_PASSWORD  
PROD_DB_PASSWORD
```

The logical configuration remains:

```
DB_PASSWORD
```

and the selected environment decides which value is used.

Using Environment Secrets in GitHub Actions

The important property is:

```
environment: development
```

Example:

```
jobs:  
  
  deploy-dev:  
    runs-on: ubuntu-latest  
    environment: development  
  
    steps:  
  
      -name: Use DB password  
      env:  
        DB_PASSWORD: ${ secrets.DB_PASSWORD }
```



```
run: |  
    echo "Deploying to development"
```

Because the job specifies:

```
environment: development
```

the reference:

```
${{ secrets.DB_PASSWORD }}
```

uses the secret configured for the **development environment**.

Conceptually:

```
environment = development  
  
    ↓  
  
secrets.DB_PASSWORD  
  
    ↓  
  
Development DB Password
```

The same workflow structure can therefore work across multiple environments while each environment keeps its own credentials.

The Limitation of Long-Lived Secrets

Secrets management solves the problem of putting credentials directly into source code.

However, another problem remains.

Suppose GitHub Actions deploys an application to AWS using:

```
AWS_ACCESS_KEY_ID
```

```
AWS_SECRET_ACCESS_KEY
```

These values can be stored securely as GitHub Secrets.

That is much safer than writing them in YAML.

But the AWS credentials themselves may remain valid for:

```
6 months  
1 year  
or until manually rotated
```

If someone obtains those credentials, they may potentially use them outside GitHub until the credentials are revoked or expire.

The architecture is still:

```
GitHub  
  |  
  | stored AWS credential  
  ↓  
AWS
```

The problem here is not the idea of GitHub Secrets.

The problem is the use of a **long-lived credential**.

OIDC Authentication

OIDC — OpenID Connect provides another approach.

Instead of giving GitHub a permanent cloud credential, the cloud provider can trust the identity of an authorized GitHub Actions workflow.

The flow becomes:

```
GitHub proves who the workflow is  
  ↓  
AWS verifies the identity
```

↓
AWS issues temporary credentials

The workflow receives credentials only when needed, and those credentials are short-lived.

Understanding OIDC with a Hotel Example

Imagine two approaches to hotel access.

Permanent-Key Approach

Hotel gives you
a permanent master key.

You keep it forever.

If the key is stolen, it remains useful until someone explicitly disables it.

Temporary-Access Approach

You prove:
"I am the guest assigned to room 504."

↓

Hotel verifies your identity

↓

Hotel gives you a temporary access card

↓

Card eventually expires

OIDC follows a similar idea.

Instead of:

GitHub stores permanent AWS credentials

we have:

GitHub proves the workflow's identity

↓

AWS verifies it

↓

AWS issues temporary credentials

What Does GitHub Prove?

AWS does not simply trust any request claiming to come from GitHub.

GitHub's OIDC token contains **claims** describing the workflow identity.

Conceptually, these claims can represent information such as:

Repository:
coderarmy/payment-service

Branch:
main

Organization:
coderarmy

Environment:
production

AWS can create a trust policy such as:

Trust GitHub

BUT ONLY IF

```
repository = coderarmy/payment-service
```

AND

```
branch = main
```

Therefore, an authorized workflow such as:

```
coderarmy/payment-service  
main
```

may receive temporary AWS credentials.

A random repository should not satisfy that trust relationship.

The key idea is:

Access is based on the verified identity of the workflow rather than possession of a permanent cloud password.

OIDC Architecture



Temporary AWS Credentials



Deployment

With this architecture:

Permanent AWS credentials stored in GitHub:

NONE

Traditional Secrets vs OIDC

Traditional Cloud Secrets

AWS generates key



Copy key into GitHub Secret



GitHub stores it



Workflow uses it

Main concern:

Long-lived credential

OIDC

Workflow starts



GitHub proves workflow identity



AWS verifies it



AWS issues temporary credentials

```
↓  
Workflow deploys  
↓  
Credentials expire
```

Core Difference

Traditional Secret	OIDC
Credential created beforehand	Credential created when needed
Stored in GitHub Secrets	No permanent cloud credential required
May be long-lived	Temporary
Workflow proves possession of secret	Workflow proves identity
Requires credential rotation	Temporary credentials expire

The broader principle is:

Prefer short-lived credentials when the external platform supports identity-based authentication.

Build Caching

Security is not the only concern in CI/CD pipelines. Build speed also matters.

Consider a Maven project.

Every build may require dependencies such as:

```
Spring Boot  
Jackson  
JUnit  
Hibernate  
MySQL Driver  
...
```

When Maven runs:

```
mvn clean package
```

it downloads required dependencies and stores them locally.

Conceptually:

```
Internet
  |
  ↓
Download Dependencies
  |
  ↓
~/.m2/repository
  |
  ↓
Build
```

This is why Maven builds on a developer's laptop often become faster after the first run.

The dependencies already exist in the local Maven repository:

```
~/.m2/repository
```

The Problem with GitHub-Hosted Runners

GitHub-hosted runners are generally fresh environments created for individual jobs.

For example:

```
Workflow Run #1

New Runner
  ↓
Download Dependencies
  ↓
Build
  ↓
Runner Removed
```

After another commit:


```
Workflow Run #2  
  
Another Clean Runner  
  ↓  
Download SAME Dependencies  
  ↓  
Build
```

Without caching, the pipeline repeatedly downloads dependencies that may not have changed.

That wastes:

- Time
- Network bandwidth
- Build minutes
- External package registry requests

This is why dependency caching is useful.

How Build Caching Works

Instead of downloading every dependency from scratch on every run:

```
Runner  
  ↓  
Check Cache  
  ↓  
Dependencies Found  
  ↓  
Restore  
  ↓  
Build
```

If the required cache exists, GitHub restores it for the runner.

After the job finishes, appropriate dependency data can be retained in GitHub Actions cache storage for future runs.

Cache Keys

A cache should not be reused blindly forever.

Suppose the Maven project originally depends on:

```
Spring Boot  
JUnit  
MySQL
```

Dependencies are cached.

Later, `pom.xml` changes and new dependencies are added:

```
Kafka  
Redis
```

The dependency configuration has now changed.

The cache system therefore needs a way to identify whether the existing cache still corresponds to the current dependency configuration.

This identifier is the **cache key**.

Conceptually:

```
Cache Key =  
Operating System + Hash of pom.xml
```

Example:

```
ubuntu-maven-98adbc12
```

Cache Hit

If `pom.xml` remains unchanged:

```
pom.xml unchanged
  ↓
Same hash
  ↓
Same cache key
  ↓
CACHE HIT
```

The existing cache can be restored.

Cache Miss

If `pom.xml` changes:

```
pom.xml changed
  ↓
New hash
  ↓
New cache key
  ↓
CACHE MISS
```

The runner downloads the required dependencies and a new cache can then be created.

```
Download Dependencies
  ↓
Create New Cache
```

So caching is not simply:

| Always reuse old files.

It is closer to:

| Reuse previously downloaded files when the cache identity matches the current dependency configuration.

Maven Caching with `setup-java`

For common Java workflows, caching does not require writing complicated caching logic manually.

`actions/setup-java` supports Maven dependency caching directly.

```
-name: Set up Java
uses: actions/setup-java@v4
with:
  distribution: temurin
  java-version: '21'
  cache: maven
```

The important line is:

```
cache: maven
```

GitHub Actions can then handle Maven dependency caching for the workflow.

One important distinction:

```
Cache data
  ↓
GitHub Actions Cache Storage
```

The cache is **not permanently stored on the individual GitHub-hosted runner**.

The runner itself is temporary.

Future runners can restore the cached dependency data from GitHub's cache storage.

What Is a Build Artifact?

A CI/CD pipeline begins with source code.

Example:

```
.java files  
pom.xml  
application.properties
```

Maven then runs:

```
mvn package
```

and may produce:

```
payment-service-1.4.2.jar
```

That generated JAR is a **build artifact**.

In simple terms:

A build artifact is an output produced by the build process that can be stored, distributed, tested, or deployed.

Examples of Build Artifacts

Different ecosystems produce different types of artifacts.

Ecosystem	Typical Artifact
Java	<code>.jar</code> , <code>.war</code>
Node.js	Package / Bundle
Python	Wheel
.NET	NuGet Package
Docker	Container Image

For a Spring Boot application:

```
Source Code  
↓  
Maven Build
```

↓
JAR File

The JAR is what ultimately matters for deployment.

Why Not Store Build Artifacts in Git?

Git repositories are mainly designed for source code and version-controlled text files.

For example:

```
GitHub  
↓  
.java  
.yml  
pom.xml
```

Build outputs have a different purpose.

We may want to:

- Store released versions
- Download an older build
- Share packages across teams
- Deploy exactly the same binary that passed testing
- Manage package versions
- Store large binary files

For these purposes, companies commonly use an **Artifact Repository Manager**.

Artifact Repository Managers

Two common examples are:

Sonatype Nexus Repository

JFrog Artifactory

A beginner-friendly mental model is:

GitHub stores source code; Nexus or Artifactory stores binaries and packages.

This is simplified, but useful for understanding their role.

```
GitHub
↓
.java
.yml
pom.xml

Nexus / Artifactory
↓
.jar
.war
.zip
npm packages
Docker images
etc.
```

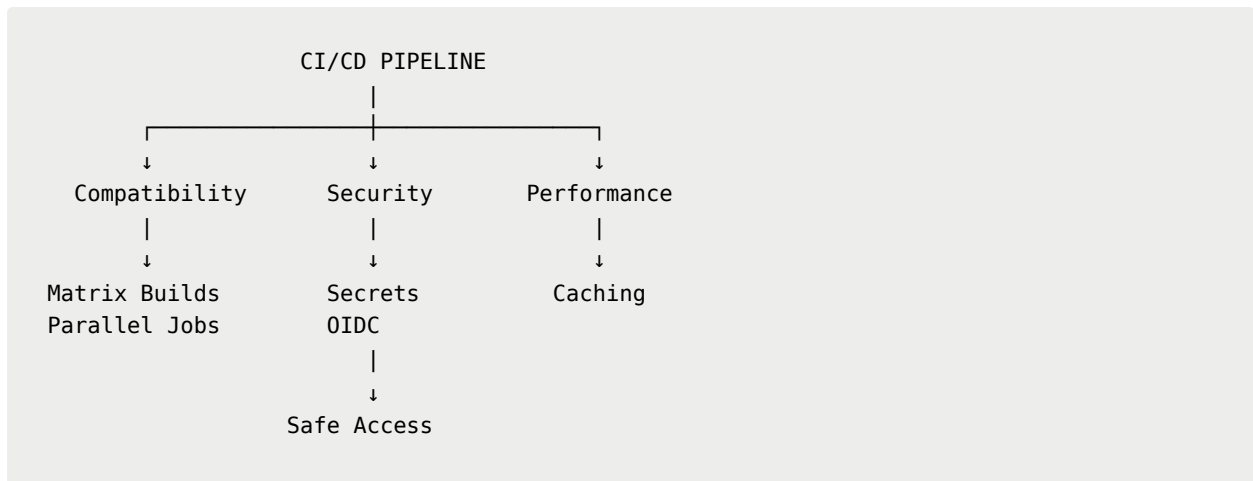
So a typical flow becomes:

```
Source Code
↓
GitHub
↓
CI Pipeline
↓
Build
↓
Artifact
↓
Nexus / Artifactory
```

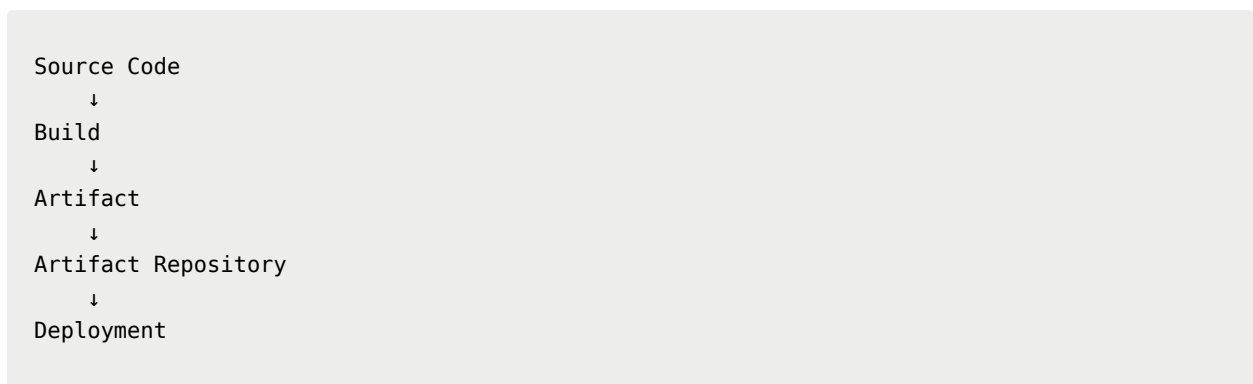
The stored artifact can later become the exact version used during deployment.

Final Mental Model

These concepts solve different CI/CD problems.



And after the application is built:



Remember

Matrix Builds

→ Run the same job across multiple configurations.

Parallel Jobs

→ Run independent work simultaneously.

Secrets

→ Keep sensitive credentials outside source code.

Environment Secrets

→ Maintain different credentials for development, staging, and production.

OIDC

→ Let workflows prove their identity and obtain temporary cloud credentials instead of storing permanent ones.

Build Caching

→ Avoid repeatedly downloading dependencies that have not changed.

Build Artifact

→ The deployable or distributable output produced by the build process.

Nexus / Artifactory

→ Repository managers used to store and manage packages and build artifacts.